SOFRIX

INTERVIEW MASTER CHEATSHEET

Muhammad Faizan Yousaf · FAST-NUCES 2026 · Full Stack Developer

React Native · MERN Stack · Next.js · PostgreSQL · System Design · Behavioural

#	Section	Covers
1	React Native Deep Dive	Core, Hooks, Navigation, Performance, BLE
2	JavaScript Fundamentals	Closures, Event loop, Async/Await, Prototypes
3	MERN Stack	MongoDB, Express, REST, JWT, RAG, Schema Design
4	Next.js & SSR	SSR/SSG/CSR, File routing, API routes, Optimisation
5	Databases	SQL, Indexes, JOINs, Transactions, NoSQL vs SQL
6	API Design & Optimisation	REST, Caching, Pagination, Rate limiting, N+1
7	System Design Basics	Microservices, Queues, Load balancing, Scaling
8	Your Projects	HealthMate, RentEase, AI-KOS, RAG Diagnostic
9	AI & ML	Whisper, FAISS, RAG, Loan prediction, LLMs
10	Git & DevOps	Git workflow, Docker, CI/CD basics
11	Behavioural	STAR answers, Experience gap, Salary
12	Questions to Ask Them	Researched, smart, shows seriousness

1. REACT NATIVE DEEP DIVE

Their #1 requirement — know this cold

Q: What is React Native? How does it differ from React.js?

React.js renders HTML elements inside a browser DOM. React Native renders actual native iOS/Android UI components — no browser involved. Both use the same component model and JSX syntax, but the output is completely different.

- **React.js:** div → browser renders HTML
- **React Native:** <View> → React Native bridge → iOS UIView / Android ViewGroup

The **JavaScript Bridge**: Your JS code runs in a JS thread. When it needs to update the UI, it serialises instructions as JSON and sends them across the bridge to the native thread. This bridge is the main performance bottleneck in older RN (pre-0.68).

New Architecture (JSI): React Native 0.71+ replaces the old bridge with JSI (JavaScript Interface) — direct C++ bindings, no JSON serialisation, much faster native calls.

Q: Core Components — name and explain each

Component	HTML Equivalent	Key Notes
View	div	Container for layout. Flexbox by default.
Text	p / span	ONLY component that can render text strings.
Image	img	Requires explicit width/height. Use resizeMode.
TextInput	input	Has onChangeText, value props. Controlled component pattern.
TouchableOpacity	button	Dims on press. Wrap anything to make it tappable.
Pressable	button	Newer, more flexible. Has pressed state styles.
ScrollView	div + overflow:scroll	Renders ALL children at once. Bad for long lists.
FlatList	virtualised list	Only renders visible items. Use for 50+ items.
SectionList	grouped list	FlatList with section headers.
Modal	dialog	Overlay. Has visible prop.
ActivityIndicator	spinner	Loading spinner. Platform native style.
SafeAreaView	—	Avoids notch/status bar on iOS. Always wrap root.

Q: FlatList vs ScrollView — when to use which?

ScrollView: renders ALL children immediately. Great for short, fixed content (settings page, form). Bad if you have 200+ items — wastes memory and causes jank.

FlatList: only renders items currently visible on screen (windowing/virtualisation). Mandatory for any dynamic data list. Key props:

- **data** — array of items
- **renderItem** — function returning JSX for each item
- **keyExtractor** — unique key per item (required, prevents re-render bugs)
- **onEndReached** — triggers when user scrolls near bottom (pagination!)
- **initialNumToRender** — how many items to render first load
- **getItemLayout** — if item height is fixed, provide this for huge perf gain

■ TIP: If FlatList items are re-rendering too much: wrap renderItem in useCallback, wrap the item component in React.memo.

Q: StyleSheet — why use it instead of inline styles?

StyleSheet.create({}) is more than syntax sugar. It validates style properties at runtime (in dev), processes the styles once and caches them as an ID reference. Inline styles create a new object on every render — wasted memory.

```
const styles = StyleSheet.create({ box: { flex: 1, backgroundColor: '#fff', padding: 16 } })
```

Q: Hooks — useState, useEffect, useRef, useCallback, useMemo

Hook	What it does	When to use
useState	Local state that triggers re-render on change	Counters, toggle, form fields, API data

useEffect	Side effects after render	API calls, subscriptions, BLE connection lifecycle
useRef	Mutable ref that doesn't trigger re-render	Storing timer IDs, previous values, direct DOM access
useCallback	Memoises a function reference	Functions passed as props to child components
useMemo	Memoises a computed value	Expensive calculations, filtered/sorted lists
useContext	Consume React Context	Auth state, theme, global settings
useReducer	Complex state logic like Redux	Multi-action state machines

Q: Props Drilling — what is it and how do you solve it?

Problem: Component A has data. Component D (deep inside B → C → D) needs it. You end up passing props through B and C even though they don't use it. This makes code brittle — add a new field and you touch 3 files.

Solutions (in order of complexity):

- **Component Composition:** Restructure so the parent directly renders the deep child with the data. No drilling needed.
- **Context API:** Built into React. Create a Context, wrap your tree in a Provider, consume with useContext anywhere. Best for auth state, theme, user preferences.
- **Zustand:** Lightweight global store. Dead simple API — create a store with state and actions, consume anywhere with useStore(). Best for medium complexity.
- **Redux Toolkit:** Industry standard for large apps. More boilerplate but very predictable. Slices, actions, selectors, async thunks.

```
// Context example const AuthContext = React.createContext(null); function App() { return } function ProfileScreen() { const user = useContext(AuthContext); }
```

Q: Navigation — Stack vs Tab vs Drawer. How do you nest them?

Library: **React Navigation**. Install @react-navigation/native + the specific navigator.

- **Stack Navigator:** Push/pop screens like a browser history. Each screen slides over the last. Press back = pop. Used for auth flows, detail screens.
- **Tab Navigator:** Bottom tabs. Screens stay mounted when you switch tabs — state is preserved. Used for main app sections.
- **Drawer Navigator:** Hamburger menu that slides in. Less common on mobile, more on tablets.
- **Nesting:** Most common pattern = Stack navigator at root (for auth), then a Tab navigator once logged in, then Stack navigators inside each tab for sub-flows.

```
// Nested: Auth Stack → Main Tabs → Each Tab has its own Stack
```

Q: React Native Performance — how do you optimise a slow app?

- **Use FlatList not ScrollView** for lists. Add keyExtractor and getItemLayout.
- **React.memo** on list item components — prevents re-render when parent re-renders.
- **useCallback** on functions passed as props — stable reference stops child re-renders.
- **useMemo** on expensive computations like filtering large arrays.
- **Avoid inline styles** — new object every render. Use StyleSheet.create().
- **Avoid anonymous functions in JSX** — same problem, new reference every render.
- **Image caching:** Use react-native-fast-image instead of Image for remote images.
- **InteractionManager.runAfterInteractions** — defer heavy work until animations finish.
- **Hermes engine** — enable it. Pre-compiles JS to bytecode, dramatically faster startup.

■ ***WATCH OUT: Never do heavy computation directly in render. Move it to useMemo or a background task.***

2. JAVASCRIPT FUNDAMENTALS

Fresher trap — they test JS depth

Q: What is a closure?

A closure is a function that remembers the variables from its outer scope even after the outer function has finished executing. The inner function 'closes over' its environment.

```
function counter() { let count = 0; return function() { count++; return count; }; } const inc = counter(); inc(); // 1 — count is remembered even though counter() is done
```

■ ***TIP: Practical use: factory functions, data encapsulation, event handlers that need to remember config.***

Q: Event loop — how does async JS work?

JavaScript is single-threaded — one thing at a time. But browsers/Node have Web APIs (timers, fetch, I/O) that run in the background. The event loop coordinates:

- **Call Stack:** Where synchronous code runs. LIFO.
- **Web API / libuv:** Handles async operations (setTimeout, fetch, fs.readFile).
- **Callback Queue (Task Queue):** Completed async callbacks wait here.
- **Microtask Queue:** Promises and queueMicrotask go here. ALWAYS drained before Task Queue.
- **Event Loop:** When call stack is empty, picks from microtask queue first, then task queue.

```
console.log('1'); setTimeout(() => console.log('3'), 0); Promise.resolve().then(() => console.log('2')); // Output: 1, 2, 3 — Promise microtask beats setTimeout macrotask
```

Q: Async/Await vs Promises vs Callbacks

- **Callbacks:** Oldest pattern. Pass a function to be called when done. Leads to 'callback hell' — deeply nested, hard to read.
- **Promises:** Object representing future value. Chain with .then().catch(). Solves nesting but long chains are still messy.
- **Async/Await:** Syntactic sugar over Promises. Write async code that reads like sync. Use try/catch for errors.

```
// Async/await example async function fetchUser(id) { try { const res = await fetch('/api/users/' + id); const data = await res.json(); return data; } catch (err) { console.error('Failed:', err); } }
```

Q: var vs let vs const

	var	let	const
Scope	Function	Block	Block
Hoisting	Yes (undefined)	Yes (TDZ error)	Yes (TDZ error)
Re-declare	Yes	No	No
Re-assign	Yes	Yes	No (value)
When to use	Never (legacy)	When value changes	Default choice

Q: Spread, Rest, Destructuring — give examples

```
// Spread: expand array/object const a = [1,2]; const b = [...a, 3]; // [1,2,3] const user = {name:'Ali'}; const full = {...user, age:25}; // Rest: collect remaining args function sum(...nums) { return nums.reduce((a,b)=>a+b, 0); } // Destructuring const {name, age=0} = user; // object, with default const [first, , third] = [1,2,3]; // array, skip second
```

Q: == vs === ?

== (loose equality): coerces types before comparing. '5' == 5 is TRUE. A source of bugs.

=== (strict equality): no coercion. '5' === 5 is FALSE. Always use ===.

■ ***SAY THIS: Always use ===. The only reason == exists is legacy code. In interviews, say 'I always use strict equality.'***

3. MERN STACK — AI Knowledge OS

MongoDB · Express · React · Node.js

Q: What is MERN? Explain each layer and how they talk to each other.

MERN is a full-stack JavaScript architecture. One language across the entire stack.

Layer	Tech	Role
Database	MongoDB	NoSQL document store. Stores JSON-like BSON documents. Flexible schema.
Backend	Express.js	Web framework for Node. Defines routes, middleware, request handling.
Runtime	Node.js	JS runtime on the server. Event-driven, non-blocking I/O. Powers Express.
Frontend	React	UI library. Component tree. Talks to backend via fetch/axios REST calls.

Q: Mongoose — what is it and why use it over raw MongoDB driver?

Mongoose is an ODM (Object Data Modelling) library for MongoDB. Raw MongoDB driver lets you insert anything — no structure enforced. Mongoose adds:

- **Schema definition:** Define the shape and data types of your documents.
- **Validation:** required, minLength, enum — rejects bad data before it hits DB.
- **Middleware (hooks):** pre/post hooks on save, find, delete — e.g. hash password before save.
- **Virtual fields:** Computed properties not stored in DB (e.g. fullName from firstName + lastName).
- **Population:** Like a JOIN — replace a reference ID with the actual document.

```
const UserSchema = new mongoose.Schema({ email: { type: String, required: true, unique: true, lowercase: true }, passwordHash: { type: String, required: true }, role: { type: String, enum: ['user', 'admin'], default: 'user' }, createdAt: { type: Date, default: Date.now } }); UserSchema.pre('save', async function() { if (this.isModified('password')) this.passwordHash = await bcrypt.hash(this.password, 10); });
```

Q: JWT Authentication — full flow, where to store, security risks

Flow: User logs in → server verifies credentials → server signs a JWT (header.payload.signature) with a secret key → client receives token → client sends it with every request → server verifies signature, reads userId.

JWT Structure: Three base64-encoded parts separated by dots. Header (algorithm), Payload (claims like userId, role, expiry), Signature (HMAC of header+payload with secret).

Storage	XSS Risk	CSRF Risk	Verdict
localStorage	HIGH — JS can read it	None	■ Never
httpOnly Cookie	None — JS can't access	Low (use CSRF token)	■ Best
Memory (JS var)	None	None	■ Good but lost on refresh

■ TIP: Access token: short-lived (15min). Refresh token: long-lived (7 days), stored httpOnly cookie. On expiry, use refresh token to get a new access token silently.

Q: RAG — Retrieval Augmented Generation — explain and your implementation

Problem LLMs have: They know only what they were trained on. They hallucinate confidently. They can't know your company's private documents.

RAG solution: Before asking the LLM, fetch relevant documents from YOUR data and inject them as context. The LLM then answers based on actual retrieved facts, not memorised training.

- **Step 1 — Ingestion:** Take your documents, split into chunks (~500 tokens). Convert each chunk to a vector embedding using Sentence Transformers. Store vector + text in DB.
- **Step 2 — Retrieval:** User asks a question. Convert question to a vector. Find top-K most similar chunks using cosine similarity / FAISS.
- **Step 3 — Generation:** Send: system prompt + retrieved chunks + user question to LLM. LLM generates answer grounded in your documents.

```
// Your AI Knowledge OS implementation // 1. Embed the query const queryVector = await sentenceTransformer.encode(userQuestion); // 2. Search MongoDB for similar document chunks const
```

```
topDocs = await Document.find().sort by cosine similarity to queryVector; // 3. Build prompt const prompt = `Context: ${topDocs.map(d=>d.content).join(' ')} Question: ${userQuestion}`; // 4. Call LLM const answer = await llm.generate(prompt);
```

Q: Express Middleware — what is it, write an example

Middleware is a function with (req, res, next) signature that runs between receiving a request and sending a response. You can chain multiple middlewares. Each calls next() to pass to the next one.

```
// Auth middleware — runs before protected routes function requireAuth(req, res, next) { const token = req.cookies.token; if (!token) return res.status(401).json({ error: 'Unauthorised' }); try { const decoded = jwt.verify(token, process.env.JWT_SECRET); req.user = decoded; // Attach user to request next(); // Continue to route handler } catch { res.status(401).json({ error: 'Invalid token' }); } } // Usage app.get('/api/projects', requireAuth, getProjectsHandler);
```

4. NEXT.JS & SERVER-SIDE RENDERING

RentEase — SSR, routing, optimisation

Q: SSR vs SSG vs CSR vs ISR — explain each with use cases

Method	When HTML is built	Data freshness	Use for	Your project
CSR	In browser, after JS loads	Realtime	Auth-gated dashboards	React SPA
SSR	On server, per request	Always fresh	User-specific pages, listings	RentEase property pages
SSG	At build time, once	Stale (rebuild to update)	Marketing, docs, blogs	RentEase landing page
ISR	Build time + revalidate	Configurable freshness	Semi-dynamic content	Next.js revalidate option

Q: File-based routing — pages/ vs app/ directory

- **pages/ (Next.js 12 and below):** pages/index.js → /. pages/[id].js → /123 (dynamic). pages/api/users.js → API route at /api/users.
- **app/ (Next.js 13+):** Uses React Server Components by default. app/page.tsx → /. app/[id]/page.tsx → /123. Layout.tsx wraps all child routes — great for persistent nav.
- **API Routes:** pages/api/*.js or app/api/*/route.ts — serverless functions. No separate backend needed for simple APIs.

```
// pages/properties/[id].js import { useRouter } from 'next/router'; export default function PropertyPage({ property }) { const router = useRouter(); const { id } = router.query; return {property.address}; } // Fetch data on server before render export async function getServerSideProps({ params }) { const res = await fetch(`${API}/properties/${params.id}`); const property = await res.json(); return { props: { property } }; }
```

Q: How did you optimise the Next.js frontend in RentEase?

- **Image optimisation:** Used next/image instead of img. Auto-resizes, lazy-loads, serves WebP.
- **Code splitting:** Next.js does this automatically. Each page is its own bundle.
- **Dynamic imports:** Heavy components (PDF viewer, rich text editor) loaded lazily with dynamic().
- **API call reduction:** Used getServerSideProps to batch fetch on server, avoiding multiple client-side calls.
- **Caching:** Set Cache-Control headers on Django responses for static-ish data (property types, cities).

5. DATABASES — SQL, NoSQL, Optimisation

Deep questions expected here

Q: PostgreSQL vs MongoDB — decision framework

Criteria	PostgreSQL (SQL)	MongoDB (NoSQL)
Data structure	Tabular, fixed schema	JSON documents, flexible schema
Relationships	FKs, JOINs — strong	Manual references or embedded docs
Transactions	Full ACID — atomic multi-table ops	Multi-doc transactions added in 4.0
Scaling	Vertical (bigger machine)	Horizontal (sharding across servers)
Best for	Finance, ERP, rental systems	Catalogs, content, embeddings, IoT
Your project	RentEase — payments, agreements	AI Knowledge OS — documents, vectors

Q: Indexes — deep explanation, types, when NOT to use

What an index is: A separate data structure (usually a B-tree) the DB maintains alongside your table. Like a book index — instead of reading every page, jump to the right page immediately.

Without index: DB does a sequential scan — O(n) time. On a 1M row table, that's reading 1M rows for every query.

With index: B-tree lookup — O(log n). Fast.

- **Single-column index:** CREATE INDEX idx_email ON users(email). Use on columns you filter or sort by.
- **Composite index:** CREATE INDEX idx_tenant_prop ON agreements(tenant_id, property_id). Order matters — most selective column first.
- **Unique index:** Enforces uniqueness + speeds lookup. Auto-created on PRIMARY KEY and UNIQUE constraints.
- **Partial index:** CREATE INDEX idx_active ON agreements(status) WHERE status='active'. Index only a subset.

When NOT to index:

- Small tables — sequential scan is fine, index overhead not worth it.
- Columns rarely queried — index costs write time without benefiting reads.
- High-write, low-read tables — every INSERT/UPDATE/DELETE must also update all indexes. Over-indexing kills write performance.
- Low-cardinality columns — indexing a boolean column (true/false) is nearly useless. DB still reads half the table.

Q: ACID — what does it mean and why does it matter?

Property	Meaning	Example
Atomicity	All or nothing — partial updates never saved	Transfer money: debit AND credit, or neither.
Consistency	DB always moves from one valid state to another	Foreign key constraint: can't add agreement for non-existent tenant.
Isolation	Concurrent transactions don't interfere	Two users booking same property don't see each other's partial writes.
Durability	Committed data survives crashes	Power cut after commit — data still there after restart.

Q: SQL Joins — explain all types with examples from RentEase

```
-- INNER JOIN: only rows with match in BOTH tables SELECT a.id, t.name, p.address FROM agreements a INNER JOIN tenants t ON a.tenant_id = t.id INNER JOIN properties p ON a.property_id = p.id WHERE a.status = 'active'; -- LEFT JOIN: all rows from left, NULLs if no right match -- Find all tenants even if they have no current agreement SELECT t.name, a.id AS agreement_id FROM tenants t LEFT JOIN agreements a ON t.id = a.tenant_id; -- Self JOIN: compare rows within same table SELECT a.address, b.address FROM properties a, properties b WHERE a.owner_id = b.owner_id AND a.id != b.id;
```

Q: N+1 Query Problem — what is it, how do you fix it?

Problem: You run 1 query to get 10 items, then 1 query PER item to fetch related data = 11 queries. With 100 items, that's 101 queries. Kills performance.

```
// N+1 (BAD) const agreements = await Agreement.find(); // 1 query for (const a of agreements) { a.tenant = await Tenant.findById(a.tenantId); // N queries! } // Fix: Eager loading / JOIN const agreements = await Agreement.find().populate('tenantId'); // 1 query with JOIN // Or in SQL: use JOIN
```

```
to fetch everything in one query
```

■ TIP: In Django: use select_related() for FKs and prefetch_related() for M2M. In Mongoose: use populate(). Always check your ORM's query count in development.

6. API DESIGN & OPTIMISATION

High-value topic for '2 year exp' interview

Q: What makes a good RESTful API?

- **Stateless:** Each request is self-contained. No server-side sessions.
- **Resource-based URLs:** Nouns, not verbs. /properties not /getProperties.
- **Correct HTTP verbs:** GET=read, POST=create, PUT=full replace, PATCH=partial update, DELETE=remove.
- **Correct status codes:** 200 OK, 201 Created, 400 Bad Request, 401 Unauth, 403 Forbidden, 404 Not Found, 422 Validation Error, 500 Server Error.
- **Consistent response shape:** Always return { data, error, meta } structure.
- **Versioning:** /api/v1/properties — so you can change v2 without breaking existing clients.

Q: Pagination — types and implementation

Essential for any list endpoint. Never return 10,000 records at once.

- **Offset-based:** ?page=2&limit;=20. Easy to implement. Problem: if items are inserted between requests, you get duplicates or miss items. Also slow on large offsets (DB scans and skips).
- **Cursor-based:** ?cursor=lastSeenId&limit;=20. More complex but consistent — no duplicates on insert. Used by Twitter, Instagram. Cursor is usually an encoded timestamp or ID.
- **Keyset pagination:** WHERE id > lastId ORDER BY id LIMIT 20. Fast — uses index. Best for sequential data.

```
// Express offset pagination GET /api/properties?page=1&limit;=20 const page = parseInt(req.query.page) || 1; const limit = parseInt(req.query.limit) || 20; const skip = (page - 1) * limit; const total = await Property.countDocuments(filter); const properties = await Property.find(filter).skip(skip).limit(limit); res.json({ data: properties, total, page, pages: Math.ceil(total/limit) });
```

Q: Caching — how do you cache API responses?

- **HTTP Cache headers:** Cache-Control: max-age=300 tells browsers/CDNs to cache for 5 minutes. Zero code needed on client.
- **Server-side cache (Redis):** Store computed results in Redis with a TTL. On request, check Redis first. If hit, return immediately. If miss, compute and store.
- **CDN caching:** For public APIs, put CloudFront/Cloudflare in front. Serve cached responses from edge — near the user.
- **Database query cache:** Mongoose/Sequelize query caching. Or PostgreSQL's built-in plan cache.

```
// Redis cache pattern async function getProperties(city) { const cacheKey = `properties:${city}`; const cached = await redis.get(cacheKey); if (cached) return JSON.parse(cached); // Cache hit! const data = await Property.find({ city }); // DB query await redis.setex(cacheKey, 300, JSON.stringify(data)); // Cache 5min return data; }
```

Q: Rate limiting — what, why, how?

What: Limit how many requests a client can make in a time window. **Why:** Prevent abuse, brute-force attacks on login, server overload. **How:** Track requests per IP/user in Redis or in-memory.

```
// Express rate limiting with express-rate-limit import rateLimit from 'express-rate-limit'; const limiter = rateLimit({ windowMs: 15 * 60 * 1000, // 15 minutes max: 100, // 100 requests per window message: { error: 'Too many requests, try again later.' } }); app.use('/api/', limiter); // Stricter on login app.use('/api/auth/login', rateLimit({ windowMs: 60000, max: 5 }));
```

Q: Input validation — how do you validate API inputs?

- **Server-side validation is mandatory.** Never trust client input. Client validation is UX only.
- **Express + Joi/Zod:** Define schema, validate req.body before it touches your DB.
- **Django REST Framework:** Serialisers validate automatically. Invalid fields return 400 with field-level errors.
- **Mongoose:** Schema validation rejects documents with wrong types or missing required fields.
- **Sanitisation:** Strip HTML from text inputs to prevent XSS. Use DOMPurify or validator.js.

```
// Zod validation in Express import { z } from 'zod'; const PropertySchema = z.object({ address: z.string().min(10).max(200), rent: z.number().positive().max(1000000), bedrooms: z.number().int().min(0).max(20), }); app.post('/api/properties', (req, res) => { const result = PropertySchema.safeParse(req.body); if (!result.success) return res.status(422).json({ errors:
```

```
result.error.flatten() }); // Safe to use result.data });
```

7. SYSTEM DESIGN BASICS

Freshie trap — they want to see you think at scale

Q: Microservices vs Monolith — explain with your RentEase example

Monolith: All functionality in one codebase, one deployment. Simple to develop and deploy early on. As it grows — harder to scale, one bug can bring everything down.

Microservices: Each feature/domain is a separate service. Independent deployment. Can scale individually. More complex infra, network latency between services.

Your RentEase example: Main Django service handled core business (properties, tenants, agreements). PDF generation was heavy and slow — separated it into a FastAPI microservice. When a user requests a PDF, Django calls FastAPI, gets the PDF, returns it. Main API stays responsive.

> ■ ***TIP: This is a real microservice decision you made. Explain WHY: PDF generation was blocking the main thread and we wanted to scale it independently and swap the PDF library without touching the core app.***

Q: What is a message queue? When would you use one?

A message queue (RabbitMQ, Kafka, AWS SQS) decouples services. Producer pushes a message. Consumer processes it asynchronously. Producer doesn't wait.

Use cases:

- Sending emails/SMS — don't block the API response to send email. Push to queue, worker sends it.
- Processing uploads — user uploads a video. API responds immediately. Worker processes/transcodes it.
- Payment webhooks — Stripe sends a webhook, push to queue, process payment confirmation reliably.
- Heavy computations — ML inference, PDF generation, report generation.

Q: How does horizontal scaling work? What is a load balancer?

Vertical scaling: Buy a bigger server (more RAM, CPU). Has a limit and is expensive.

Horizontal scaling: Add more servers running the same app. Cheaper, no limit. But requests must be routed intelligently.

Load balancer sits in front of your servers. Distributes incoming requests across them. Algorithms: round-robin, least-connections, IP hash (same user always hits same server — useful for sessions).

Challenge with horizontal scaling: Stateless apps scale easily. If you store sessions in server memory, users get logged out when hitting a different server. Solution: store sessions in Redis (external, shared state).

Q: What is Docker? Why did you use it in HealthMate?

Docker packages an application and all its dependencies into a container — a lightweight, isolated environment that runs consistently on any machine.

- **Problem it solves:** 'Works on my machine' — dependencies, OS differences, Python versions. Docker eliminates this.
- **Image:** Blueprint (like a class). Built from Dockerfile.
- **Container:** Running instance of an image (like an object).
- **docker-compose:** Defines and runs multi-container apps. In HealthMate: one container for Flask API, one for MySQL, one for the app. One command starts everything.

```
# docker-compose.yml example services: api: build: ./backend ports: ['5000:5000'] environment: DB_URL: mysql://db:3306/healthmate depends_on: [db] db: image: mysql:8 environment: MYSQL_DATABASE: healthmate MYSQL_ROOT_PASSWORD: secret
```

8. YOUR PROJECTS — Own every line

Say 'I built' not 'we built'

HealthMate FYP

Q: Walk me through HealthMate. What exactly did YOU build?

> ■ ***SAY THIS: I built the complete mobile frontend in React Native with Expo. That includes: the authentication flow (login, register, JWT token storage), the dashboard screen with real-time health metric visualisation, the wearable device BLE pairing flow using react-native-ble-plx, and the REST API integration with our Flask + MySQL backend.***

Q: How did BLE work exactly? What was the hardest part?

- **BLE flow:** Scan for nearby devices → user selects their wearable → connect → discover services → subscribe to health data characteristics (heart rate, steps) → receive notifications when data changes.
- **Library:** react-native-ble-plx. Couldn't use Expo's managed workflow for raw BLE — ejected to bare workflow.
- **Hardest part:** BLE is asynchronous and unreliable. Devices drop, data arrives at irregular intervals, byte format for heart rate is a specific bit-packed structure.
- **Solution:** useEffect to manage connection lifecycle with cleanup on unmount. useRef to store device reference. Retry logic on disconnect. Buffer + parse BLE byte arrays into usable numbers.

RentEase — Full Stack SaaS

Q: This is your most complex project — walk through it technically.

The problem: Pakistan's rental market is informal — paper agreements, verbal deals, no tracking.

What I built: Full-stack SaaS. Next.js frontend with Tailwind, Django REST Framework backend, PostgreSQL database, FastAPI microservice for PDF generation using ReportLab.

- **Database design:** Tenants, Properties (with owner FK), Agreements (tenant FK + property FK), Payments (agreement FK), Notices. All with proper FK constraints and indexes on foreign keys.
- **Django REST Framework:** ViewSets + Serialisers for each model. JWT auth via djangorestframework-simplejwt. Permissions: property owners can only manage their own properties.
- **FastAPI PDF service:** Separate process on a different port. Django calls it via HTTP when a PDF is requested. Stateless — just receives agreement data, returns PDF bytes. ReportLab builds the template.
- **Next.js frontend:** SSR for property listing pages (SEO + fresh data). SSG for landing page. Client-side for the dashboard. File-based routing, getServerSideProps for data fetching.

AI Knowledge OS — MERN

Q: Explain the RAG implementation specifically.

- Users create projects and upload documents into them.
- On upload, document is split into 500-token chunks. Each chunk is embedded using Sentence Transformers (768-dimensional vector). Stored in MongoDB alongside the original text.
- On question: embed the question → cosine similarity search over all document vectors → retrieve top 5 chunks → inject as context into LLM prompt → return grounded answer.
- Security: JWT auth ensures users only search within their own projects' documents.

9. AI & ML PROJECTS

Whisper, FAISS, Loan prediction

Q: Explain your Whisper pipeline at Digital Genei

- **Problem:** Hours of meeting recordings with important decisions buried inside.
- **Pipeline:** Audio file → OpenAI Whisper (speech-to-text) → raw transcript → NLP summarisation (extractive or abstractive) → structured summary with key decisions.
- **FAISS:** Facebook AI Similarity Search. After transcription, embed each meeting's summary. Store in FAISS index. User can semantically search past meetings — 'what did we decide about the API contract?' returns the relevant meeting, not keyword matches.
- **Result:** 60% reduction in manual review time across 50+ sessions.

■ TIP: If asked about summarisation: explain extractive (pick key sentences) vs abstractive (generate a summary). Say you used an open-source model or Hugging Face pipeline.

Q: Loan prediction — explain your ML pipeline

- **Dataset:** 10,000+ loan application records. Features: income, credit score, employment status, loan amount, tenure, existing debt.
- **Pipeline:** (1) Data cleaning — handle missing values (mean imputation for continuous, mode for categorical). (2) EDA — find correlations, check class imbalance. (3) Feature engineering — debt-to-income ratio, credit score bucket. (4) Encode categoricals — one-hot or label encode. (5) Train/test split 80/20. (6) Train model. (7) Evaluate.
- **Models compared:** Logistic Regression (baseline, interpretable), Random Forest (ensemble, handles non-linearity). Random Forest won — 87% accuracy.
- **Metrics:** Accuracy isn't enough for imbalanced data. Also reported Precision, Recall, F1, AUC-ROC. Precision = of predicted approvals, how many were real. Recall = of real approvals, how many did we catch.

Q: RAG for Clinical Notes — MIMIC-IV

- Built a clinical Q&A; system over MIMIC-IV (anonymised hospital records dataset).
- Clinical notes chunked and embedded with Sentence Transformers. Indexed with FAISS for sub-second retrieval.
- Query → embed → FAISS search → top-K relevant records → LLM generates diagnostic reasoning grounded in patient records.
- Streamlit interface — clinician types a diagnostic question, gets evidence-grounded answer with source records.
- Reduced manual chart lookup from minutes to under 6 seconds per query.

10. GIT & DEVOPS

Expected for any dev role

Q: Git workflow — how do you use Git on a team project?

- **main/master:** Always deployable. Never push directly.
- **Feature branches:** git checkout -b feature/add-payment-tracking. One branch per feature.
- **Commits:** Small, atomic, descriptive. 'Add payment model and migration' not 'stuff'.
- **Pull Request:** Push branch → open PR → team reviews → merge to main.
- **Merge conflicts:** When two branches edit the same line — resolve manually, test, commit.

```
git checkout -b feature/ble-integration # ... make changes ... git add -p # Stage interactively git commit -m 'feat: add BLE device scanning and characteristic subscription' git push origin feature/ble-integration # Open PR on GitHub
```

Q: Common Git commands you use daily

Command	What it does
`git stash`	Temporarily shelve uncommitted changes
`git stash pop`	Restore stashed changes
`git rebase main`	Replay your branch commits on top of latest main
`git cherry-pick <hash>`	Apply a single specific commit to current branch
`git log --oneline`	Compact commit history
`git diff --staged`	See what you're about to commit
`git reset --soft HEAD~1`	Undo last commit, keep changes staged
`git reset --hard HEAD~1`	Undo last commit AND discard changes. Dangerous!
`git bisect`	Binary search through commits to find which one introduced a bug

11. BEHAVIOURAL — STAR Answers

Prepare these word-for-word

Q: You only have 2 months internship. We need 2 years.

■ *WATCH OUT: Do NOT start with 'I know I lack experience but...' — you lose before you finish.*

■ *SAY THIS: "I don't have 2 years on a payroll. But I've been building production systems continuously for 2 years — a mobile app with real BLE wearable integration as my FYP, two internships where I shipped working ML and NLP systems, and full-stack projects using your exact stack: React Native, Next.js, Node.js, PostgreSQL, MongoDB. I can read a real codebase and contribute from week one. The question isn't years on a payroll — it's whether I can ship. My GitHub shows I can."*

Q: What was the hardest bug you ever solved?

■ *SAY THIS: "In HealthMate, BLE characteristic notifications were dropping silently after about 20 minutes. No error, no crash — data just stopped. I spent two days on it. Isolated it to a specific Android BLE power management behaviour — the OS was terminating the BLE connection to save battery. I had to implement a foreground service to keep the connection alive on Android, plus a reconnect handler on disconnect. Learned to always check platform-specific constraints, not just the library docs."*

Q: Walk me through your most complex project.

■ *SAY THIS: "RentEase. Pakistan's rental market runs on paper agreements and WhatsApp. I built a full-stack SaaS to digitise it. Next.js frontend, Django REST backend, PostgreSQL database with properly modelled relationships — tenants, properties, agreements, payments. The most interesting architectural decision was separating PDF generation into a FastAPI microservice. Agreement PDFs are slow to generate and I didn't want them blocking the main API. So Django calls the FastAPI service async, gets the PDF bytes, returns to the user. Main API stays fast. I used ReportLab for the PDF templates. End-to-end: list a property, sign a digital agreement, track payments, send notices."*

Q: How do you handle being stuck on a bug?

■ *SAY THIS: "Step one: reproduce it in the smallest possible case — strip everything until only the bug is left. Step two: read the error and logs carefully — I don't assume, I let the output tell me where to look. Step three: check official docs and GitHub issues for that library. Most bugs exist in someone else's issue. Step four: if I'm stuck past 30-45 minutes, I flag it — ask a colleague or post a specific question with a minimal reproducible example. I don't sit on blockers quietly. I flag early so the team can plan around it."*

Q: Why Sofrix? Why this role?

■ *SAY THIS: "I looked at your portfolio — Parrot AI and Per Diem. You're building AI-first products, not just CRUD apps. That matches exactly what I've been building — RAG systems, NLP pipelines, ML integrations on top of full-stack apps. I want to work on products that use AI to solve real problems, not just as a toy. Sofrix is doing that at production scale. And I want to work in an environment where I can ship fast and learn from senior engineers who've done it at scale."*

Q: What's your expected salary?

■ *TIP: Research market rate for junior full-stack in Faisalabad before the interview. A common range for fresh FAST-NUCES grads is 60,000–100,000 PKR depending on company.*

■ *SAY THIS: "I'm flexible — I'm prioritising the right role and growth over a specific number. Based on my research for junior full-stack in Faisalabad, I'd expect somewhere in the market range. What's the budgeted range for this position?" — Then negotiate from what they say.*

12. QUESTIONS TO ASK THEM

This part shows you did research — critical

Ask 2-3 of these. Shows seriousness, intelligence, and that you thought about the role.

Question	Why it matters
"Is this role primarily client project work or do you have internal product?"	The site shows Parrot AI and Per Diem. Know if you're building product or doing clien
"Parrot AI and Per Diem are on your portfolio — is that the type of work I'd be doing?"	Shows you researched them. Tests if the JD matches reality.
"How does a typical project work here — who handles requirements and design?"	Agencies often put PM + design + dev on one junior. Know before you join.
"What does the first 30 days look like for this role?"	Reveals if they have structured onboarding or just throw you into the deep end.
"Is the team here building for US clients?"	Their address says Santa Monica CA. You're likely the offshore dev team. Know the d
"What does the tech stack look like on active projects — is it the same as the JD?"	JDs often list aspirational tech. Find out what you'll actually use day one.
"How do senior engineers give feedback to juniors here?"	Reveals the mentorship culture. Tells you how fast you'll grow.

FINAL RULES

1. Say 'I built' not 'we built' for the parts you did. Own your work.

2. When weak on something: 'I haven't used X in production but I understand the concept and can get up to speed fast. Here's what I know...'

3. Don't over-apologise for lack of 2 years. Redirect to what you have built.

4. Pause before answering hard questions. Thinking is a sign of depth, not weakness.

5. If you don't know: say 'I haven't encountered that specific case, but my approach would be...' — never guess and bluff.